웹 화 면 명 세 서

고가용성 가상 암호화폐 거래소 프론트엔드 구현 명세

Web Screen Implementation Specification · Next.js App Router · 고정형 데스크톱

문서 버전	v2.0	작성일	2026-07-22
문서 성격	**구현 계약**. 데스크톱 트레이딩 화면(`/exchange/[market]`) 기준으로 라우트·상태·데이터 바인딩·이벤트를 확정		
대상 코드	`프로젝트/web` (Next.js App Router · TypeScript · zustand · lightweight-charts · 정적 export)		
참조 문서	데스크톱 레이아웃 설계 v1.1 · 프론트 디자인시스템 · 백엔드 API 설계 v2.5 · DB 설계 v2.5		

목차

1. **개요**. 문서 목적 · 산출물 관계 · 화면 정책
2. **라우팅 명세**. 실제 라우트 · 정적 export 리다이렉트
3. **공통 아키텍처**. 앱 프레임 · 상태관리(zustand) · DataSource · 실시간 파이프라인
4. **공통 컴포넌트 인벤토리**
5. **화면별 명세**. 거래 화면 · 자산 · 거래내역 · 시스템 상태 · 온보딩
6. **Spring 백엔드 연동 전환 계획**

1. 개요

1.1 문서 목적

본 문서는 프론트엔드 구현 계약을 고정한다. 데스크톱 레이아웃 설계 v1.1이 "어떻게 보이는가"(그리드·패널·색 규약)를 정의했다면, 본 문서는 "**어떻게 동작하고 무엇과 연결되는가**"(라우트·컴포넌트 트리·상태·데이터 바인딩·이벤트·백엔드 전환 지점)를 정의한다.

1.2 화면 정책 (고정형 데스크톱 전용)

항목	규칙
기준 화면	업비트 동형의 고정형 데스크톱 웹. 거래 화면 `/exchange/[market]` 가 메인. 차트·호가·체결·주문·마켓 목록을 한 화면에 동시 노출
최소 폭	`body` min-width 1280px. 더 좁으면 레이아웃을 접지 않고 가로 스크롤. 반응형 리플로우·모바일 레이아웃 없음
최대 폭	거래 화면 콘텐츠 max-width 1920px 중앙 정렬. 서브 화면 max-width 1200px 중앙 정렬
세로 대응	가로 미디어 쿼리 없음. 세로만 높이 브레이크포인트(820·640) 적용(레이아웃 설계 §2.3)
전 화면 프레임	상단 `HeaderNav` 1줄(56px, sticky). 하단 TabBar·모바일 AppBar·BottomSheet·`.app-frame` 은 폐지

1.3 산출물 관계

문서	관점	답하는 질문
데스크톱 레이아웃 설계 v1.1	디자인. 그리드·패널 스펙·색·접근성	화면이 어떻게 보이는가
웹 화면 명세서 v2.0 (본 문서)	구현. 라우트·상태·바인딩·이벤트·연동	화면이 어떻게 동작하는가
백엔드 API 설계 v2.5	계약. REST/WS 스키마·멱등·서명	무엇을 주고받는가

1.4 핵심 설계 결정

> **DataSource 추상화**. 모든 화면은 `DataSource` 인터페이스만 소비한다. 현재 구현 `SimDataSource` (클라이언트 GBM 시뮬레이터, 런타임 외부 의존 0)를 `HttpStompDataSource` (Spring REST + STOMP)로 **화면 코드 수정 없이** 교체한다(§6). `getDataSource()` 팩토리

가 `NEXT_PUBLIC_DATASOURCE=sim|http` 로 구현체를 선택하며 기본은 sim이다.

정적 export. `next.config.ts` 가 `output:"export"` . 산출물 `out/` 을 Nginx, Spring 정적 리소스로 서빙한다. 서버 `redirects()` · 서버 리다이렉트가 동작하지 않으므로 모든 경로 이관은 클라이언트 `router.replace` 로 처리하고, 동적 라우트는 `generateStaticParams` 로 전 마켓 경로를 사전 생성한다.

1.5 변경 이력

버전	일자	내용
v1.0	2026-07-21	초판. 모바일 앱 프레임(420px) 기준 SCR-01~07(`/market` · `/coin` · `/order`), TabBar·BottomSheet
v2.0	2026-07-22	고정형 데스크톱 전환 반영 전면 재작성. 거래 화면 `/exchange/[market]` 단일 통합, 구 라우트 클라이언트 리다이렉트, 실제 컴포넌트 트리(`components/trade/*`)·zustand 스토어· `HttpStompDataSource` 연동 반영

2. 라우팅 명세

2.1 라우트 표

경로	페이지 파일	파라미터 / 쿼리 · 동작
`/exchange/[market]`	`app/exchange/[market]/page.tsx` + `ExchangeMarketClient.tsx`	`market` : 마켓 코드(`KRW-BTC`). **거래 화면 본체.** `generateStaticParams` 로 전 마켓 사전 생성. 쿼리 `side=bid|ask` · `price` 는 OrderPanel 프리필로 승계
`/exchange`	`app/exchange/page.tsx`	클라이언트 `router.replace` . localStorage `exchange.recentMarket` (없으면 `KRW-BTC`)로 이동
`/`	`app/page.tsx`	클라이언트 `router.replace('/exchange')`
`/welcome`	`app/welcome/page.tsx`	온보딩(정적). 두 CTA 모두 `/exchange` . 시뮬레이션 데모라 실제 계정·인증 없음
`/market`	`app/market/page.tsx`	[레거시] 딥링크 보존. 클라이언트 `router.replace('/exchange')`
`/coin/[market]`	`app/coin/[market]/page.tsx` + `CoinRedirect.tsx`	[레거시] `generateStaticParams` 유지. `router.replace('/exchange/[market]')`
`/order/[market]`	`app/order/[market]/page.tsx` + `OrderRedirect.tsx`	[레거시] `generateStaticParams` 유지. `useSearchParams` 로 `side` · `price` 를 읽어 `/exchange/[market]?side=&price=` 로 승계. Suspense 경계 필수
`/assets`	`app/assets/page.tsx`	자산 화면(§5.2, 와이드)
`/history`	`app/history/page.tsx`	거래내역 화면(§5.3, 와이드)
`/system`	`app/system/page.tsx`	시스템 상태 화면(§5.4, 와이드)

마켓 코드는 API §2-1의 `KRW-BTC` (quote-base). 존재하지 않는 마켓은 거래 화면에서 EmptyState + [거래소로 돌아가기]로 안내한다.

2.2 정적 export 리다이렉트 전략

> `output:"export"` 는 서버 리다이렉트를 지원하지 않는다. 구 라우트는 **페이지 파일과 `generateStaticParams` 를 유지**한 채 본문을 클라이언트 리다이렉트 컴포넌트로 교체해 딥링크를 보존한다. `/order` 는 서버 redirect로 쿼리 승계가 불가하므로 반드시 `useSearchParams` 클라이언트 처리로 `side` · `price` 를 넘긴다. `useSearchParams` 를 쓰는 컴포넌트는 정적 프리렌더에서 `Suspense` 경계로 감싼다.

2.3 통합 배경

구 `/market` (목록)· `/coin/[market]` (상세)· `/order/[market]` (주문)을 거래 화면 하나로 통합했다. 마켓 목록·주문 폼이 거래 화면의 상주 패널이 되어 독립 페이지의 존재 이유가 사라졌다. 업비트 `/exchange` 단일 화면 모델과 동형이다.

3. 공통 아키텍처

3.1 앱 프레임 · 테마 · 폰트

항목	규칙
전역 셸	`app/layout.tsx` 가 `HeaderNav` (전 화면 공통 상단)와 `Toaster` 를 상주 렌더. 화면 본문은 `{children}` . `.app-frame` ·TabBar 없음
테마	`tokens.css` 계약: 라이트 `:root` / 다크 `[data-theme="dark"]` + `prefers-color-scheme` 미러. `ThemeToggle` 이 `data-theme` 토글. FOUC 방지 인라인 스크립트가 first paint 전 저장 테마 적용
폰트	Pretendard npm self-host(`pretendard/dist/web/variable`). CDN 금지. 런타임 외부 의존 0
스타일	Tailwind 금지. `styles/tokens.css` + CSS Modules. 하드코딩 hex 금지(코인 `brandColor` 데이터만 예외)
숫자	`.num` (tabular-nums) + `lib/format.ts` . 금액·수량은 문자열 유지(NUMERIC(40,18), `parseFloat` 는 표시용 `toNum` 에서만). 방향은 색 단독 금지. 부호(U+2212)+화살표(▲▼) 병기 + aria 라벨

3.2 상태관리. `lib/store.ts` (zustand)

상태 (ExchangeState)

필드	타입	내용
`markets`	`Market[]`	마켓 목록 (초기 1회 로드)
`tickers`	`Record<market, TickerMessage>`	마켓별 최신 시세 (WS 갱신)
`orderbooks`	`Record<market, OrderbookMessage>`	마켓별 호가 스냅샷+실시간
`trades`	`Record<market, TradeMessage[]>`	최근 체결 (마켓별 최대 60건 링버퍼)
`assets`	`Asset[]`	보유 자산 (myAsset로 갱신)
`openOrders` / `closedOrders`	`Order[]`	미체결/체결 주문 (myOrder 전이 반영)
`favorites`	`string[]`	관심 마켓. localStorage `exchange.favorites` 영속(SSR 안전 하이드레이션)
`connected`	`boolean`	WS 연결 상태 → HeaderNav 연결 점. 갱신 주체는 `DataSource.onConnectionChange` 뿐

액션

액션	동작
`apply(msg: WsMessage)`	WS 메시지 타입별 라우팅. ticker/orderbook/trade/myAsset을 슬라이스에 흡수. myOrder는 상태별 미체결↔체결 목록 전이(OPEN·PARTIAL은 openOrders, 그 외는 closedOrders로 이동). candle은 미보관(차트가 로컬 구독)
`setConnected(v)`	연결 점 갱신의 유일 경로. `onConnectionChange` 콜백이 호출
`toggleFavorite(market)`	관심 토글 + localStorage 반영

`hydrateFavorites()`	마운트 후 localStorage 값 반영 (SSR 불일치 방지)
`loadMarkets()` / `loadAccounts()` / `loadOrders(market?)`	DataSource REST 미러 호출로 초기/갱신 스냅샷 적재

구독 훅 `useLiveChannels(destinations: string[])`. 화면이 필요한 채널만 선언하면 마운트 시 구독·언마운트 시 해제. 수신 메시지는 전부 `apply()` 경유. 화면은 selector로 필요한 조각만 구독(셀 단위 리렌더 최소화).

3.3 DataSource 인터페이스. `lib/api/datasource.ts`

메서드	API v2.5 대응 (Spring 전환 시)	사용처
`getMarkets()`	`GET /api/v1/markets`	store.loadMarkets
`getTicker(markets)`	`GET /api/v1/ticker?markets=…`	초기 스냅샷 합성
`getOrderbook(market)`	`GET /api/v1/orderbook`	스냅샷·재동기화
`getTrades(market, count=30)`	`GET /api/v1/trades`	TradesPanel 백필
`getCandles(market, unit, count=120)`	`GET /api/v1/candles/{unit}`	CandleChart 백필
`getAccounts()`	`GET /api/v1/accounts`	store.loadAccounts
`getOpenOrders(market?)` / `getClosedOrders(market?)`	`GET /api/v1/orders/open|closed`	store.loadOrders
`placeOrder(req: OrderRequest)`	`POST /api/v1/orders` . **client_order_id 필수**(멱등)	OrderPanel
`cancelOrder(idOrClientId)`	`DELETE /api/v1/orders/{id}`	거래내역 취소
`subscribe(destination, cb) → unsubscribe`	STOMP SUBSCRIBE. §5-3 경로 (`dest` 헬퍼). **구독 즉시 snapshot 1건**(§5-2)	useLiveChannels
`onConnectionChange(cb) → off`	연결 상태 통지. 등록 즉시 현재 상태 1회 통지	HeaderNav
`connect()` / `disconnect()`	STOMP CONNECT / DISCONNECT	싱글턴 수명주기(`getDataSource`)

`SimDataSource` 는 `onConnectionChange` 등록 시 `true` 1회만 통지(네트워크 단절 없음). `getDataSource()` 는 클라이언트 싱글턴이며 최초 접근 시 `connect()` 1회 호출.

3.4 실시간 파이프라인

```
SimEngine (GBM 틱, 시드 PRNG)                ← 현재: 클라이언트 시뮬레이터
   │ WsMessage {type, market, seq, epoch, stream:"snapshot"|"realtime", …}
   ▼
DataSource.subscribe(dest, cb) ──→ store.apply(msg) ──→ 화면 selector 구독
                                                          │ (셀 단위 리렌더 최소화)
                                                          ▼
                                              TickCell: 유의미 변동 시 data-tick 플래시
                                              (throttle 300ms · reduced-motion 시 생략)
```

규약	구현
채널별 `seq`	단조 증가 검사. gap 감지 시 REST 스냅샷 재호출로 재동기화. HttpStompDataSource가 담당(§6)

`epoch`	변경(백엔드 페일오버) 감지 시 전 채널 강제 재동기화. 사용자 개입 없음
스냅샷 계약	구독 즉시 `snapshot` 1건 수신 후 `realtime` 스트림. 어댑터가 raw `seq/epoch/stream`을 흡수하고 store에는 연속화된 도메인 이벤트만 전달
레이아웃 시프트	실시간 갱신은 시프트 0. 호가는 10+10 슬롯 고정(빈 슬롯은 자리행). 틱 플래시는 셀별 throttle 300ms

3.5 시뮬레이터. `lib/sim/engine.ts`

GBM 틱 엔진(시드 PRNG, 재현 가능) · 코인 6종(`COIN_META` : BTC/ETH/SOL/XRP/DOGE/ADA) · 호가 10단계 · 체결 스트림 · 캔들 백필 · 주문 시뮬(멱등 `client_order_id` , 최소주문 5,000원, 잔고 락/해제, 수수료 0.0005, myOrder/myAsset 발행) · `priceStep(price)` · `CANDLE_UNIT_MS` 유틸 노출. API v2.5의 WS 메시지 스키마·채널별 독립 seq를 그대로 미러. **백엔드가 생겨도 화면이 받는 메시지 형태는 동일**.

4. 공통 컴포넌트 인벤토리

4.1 전역 · 레이아웃

컴포넌트	핵심 Props	역할
`HeaderNav`	없음 (경로 기반 active)	전 화면 공통 상단 내비(56px sticky). 로고·링크 4개(거래소·자산·거래내역·시스템 상태)·WS 연결 점· `ThemeToggle` . `onConnectionChange` 구독으로 연결 상태 갱신
`TradeLayout`	`summary/chart/book/trades/order/marketList`	거래 화면 그리드 컨테이너. grid-template + 세로 높이 브레이크포인트(820·640)만 담당, 패널은 슬롯으로 수신
`PanelCard`	`title?/actions?/dense?/scroll?`	거래 화면 패널 공통 셸. surface 카드 + 선택적 헤더 + 내부 스크롤 슬롯. `dense` =테이블성 패널 12px 패딩
`DataTable<T>`	`columns/rows/rowKey/defaultSort/rowHeight/onRowClick`	자산·거래내역·시스템 로그 공용 정렬 테이블. 헤더 sticky(top 56px), `tabular-nums` , 키보드 행 활성

4.2 거래 화면 패널 (`components/trade/*`)

컴포넌트	역할
`SummaryBar`	종목 식별(SymbolChip·한글명)·마켓 전환 드롭다운·현재가(TickCell)·등락(PricePill)·24H 스탯(고가·저가·거래대금·거래량)·관심 토글
`ChartPanel`	주기 탭(SegmentedTab, 1분~1주) + `CandleChart` (`next/dynamic` , ssr:false, height 100%)
`OrderbookPanel`	세로 호가. 매도 10호가(Blue 틴트) 위 / 현재가 경계 / 매수 10호가(Red 틴트) 아래. 잔량 막대·가격·등락률(전일종가 역산). 클릭 시 가격만 프리필(side 유지)
`TradesPanel`	최근 체결 4열 테이블(체결가·체결량·등락률·시각). sticky 헤더 + 내부 스크롤. 스토어 60건 + REST 백필 병합·중복 제거
`OrderPanel`	인라인 주문 폼. 매수(Red)·매도(Blue), 지정가·시장가, 가격 ±스텝, 수량 + 퍼센트 칩, 총액·수수료·주문가능. 인라인 2단계 확인
`MarketListPanel`	검색(코인명·심볼) + 탭(원화·관심·보유) + 정렬 헤더(한글명·현재가·전일대비·거래대금) + 코인 행. 행 클릭 시 `/exchange/[market]` 이동
`derive.ts`	파생 계산 유틸. 전일종가 역산(`prevCloseOf`), 임의 가격 등락률(`rateVsPrevClose`), 호가 실재 슬롯 판정 (`isPresentLevel`)

4.3 프리미티브

컴포넌트	핵심 Props	역할

`SegmentedTab`	`items/value/onChange/variant`	탭 전환. `directional`=매수Red/매도Blue, `neutral`, `underline`
`PricePill`	`change/changeRate/amount/variant`	등락 표시. 색+부호+화살표 병기, EVEN은 무채색
`TickCell`	`value/format`	실시간 숫자 셀. 플래시 throttle 300ms, reduced-motion 대응, memo
`SymbolChip`	`meta/size`	코인 심볼 칩(브랜드색 예외 허용 지점)
`CandleChart`	`market/unit/height`	lightweight-charts 래퍼. 양봉 Red/음봉 Blue(토큰 주입), `series.update` 실시간. `height: number | string`
`Skeleton`	`variant/width/height`	로딩 형상. CLS 0
`Toast` / `Toaster`	`toast(msg, variant)`	접수/체결/실패 알림. `info|success|error`
`ThemeToggle`	없음	`data-theme` 라이트/다크 토글
`ListRow`	`leading/title/subtitle/trailing`	범용 리스트 행 프리미티브

폐지 컴포넌트: `AppBar` · `TabBar` · `BottomSheet` (모바일 전용). 데스크톱 전환으로 제거, 죽은 코드로 남기지 않는다.

5. 화면별 명세

5.1 거래 화면. `/exchange/[market]`

컴포넌트 트리

```
app/exchange/[market]/page.tsx        (서버 셸 · generateStaticParams: 전 마켓)
└ ExchangeMarketClient                (Suspense 경계 · useSearchParams 프리렌더 대비)
   └ ExchangeMarketBody               (useParams·useSearchParams · 채널 배선·프리필 상태)
      └ TradeLayout                   (그리드 6슬롯 · 높이 브레이크포인트)
         ├ SummaryBar        (summary)
         ├ ChartPanel        (chart)   ── SegmentedTab + CandleChart(dynamic)
         ├ OrderbookPanel    (book)    ── onPricePrefill → OrderPanel
         ├ TradesPanel       (trades)
         ├ OrderPanel        (order · key=market)
         └ MarketListPanel   (marketList)
HeaderNav는 app/layout.tsx에서 전 화면 공통 렌더
```

채널 배선 · 초기 로드 (ExchangeMarketBody)

배선	내용
`useLiveChannels`	`[dest.orderbook(market), dest.trade(market), ...COIN_META.map(dest.ticker)]`. 현재 마켓 호가·체결 + 마켓 목록/요약용 전 마켓 티커
마운트 effect	`hydrateFavorites()` · 미로드 시 `loadMarkets()` · `loadAccounts()` · `loadOrders()`
최근 마켓	유효 마켓 진입 시 `writeRecentMarket(market)` 로 localStorage 갱신(`/exchange` 복원용)
OrderPanel 별도	`useLiveChannels([dest.myAsset])` + `subscribe(dest.myOrder)` 직접 구독(체결 Toast)

데이터 바인딩

패널	스토어/원천	파생
SummaryBar	`tickers[market]` · `favorites`	방향색·등락, `formatCompactKRW` 축약
OrderbookPanel	`orderbooks[market]` + `tickers[market]`	등락률 = 전일종가 역산(`prevCloseOf` + `rateVsPrevClose`). 잔량 막대 = 최대 잔량 대비 %
TradesPanel	`trades[market]` (MAX 60) + `getTrades` 백필	체결가 방향색, 등락률 역산, 중복 `trade_id` 제거
ChartPanel	`getCandles(market, unit)` 백필 + `trade` 실시간 `series.update`	주기 로컬 상태(기본 15m)
OrderPanel	`tickers[market]` · `assets` · `markets`	총액·수수료(0.05%)·주문가능, 퍼센트 칩 자동 계산
MarketListPanel	`tickers` · `favorites` · `assets`	탭 필터·정렬(거래대금 desc 기본)

이벤트

이벤트	처리

마켓 드롭다운·마켓 행·보유 행 클릭	`router.push('/exchange/[market]')`. 마켓 전환 시 이전 마켓 프리필 파기
호가 행 클릭	`onPricePrefill(price)` → OrderPanel에 **가격만 주입, side 유지**(데스크톱 상주 폼 컨텍스트 보호). 시장가 상태면 지정가로 전환
관심 토글	`toggleFavorite(market)`
주기 탭	로컬 `unit` 변경 → `getCandles` 재백필
쿼리 승계	진입 쿼리 `side`는 초기 방향, `price`는 초기 프리필로 반영

OrderPanel. 주문 검증 · 제출 규약

항목	규칙
주문 유형	지정가(`limit`) / 시장가(`market`). 시장가 매수는 `price` 에 주문 금액(notional), 시장가 매도는 `volume`
가격	지정가만 입력. ±스텝은 `priceStep(price)` 호가 단위. 프리필·현재가 1회 채움
수량 / 금액	퍼센트 칩(10·25·50·최대). 매도는 보유 수량, 지정가 매수는 가용 KRW÷(가격·수수료), 시장가 매수는 금액 기준 자동 계산. 소수는 `qty_scale` 내림
검증	최소주문 5,000원 미만 → 에러 + CTA disabled / 매수 총액+수수료 > 가용 KRW → "주문 가능 금액 초과" / 매도 수량 > 보유 → "보유 수량 초과"
멱등	`client_order_id` 클라이언트 생성(`web-{ts}-{rand}`). 확인 진입(arm) 시 1개 발급, 주문 1건당 1키
제출	`placeOrder(req)` → 접수 Toast(info) + 입력 초기화 + `loadOrders()` . 실패 시 오류 메시지 Toast(error)
체결	`myOrder` 구독에서 `status==="FILLED"` 수신 시 체결 Toast(success)

> **인라인 2단계 확인**. 1차 클릭 시 CTA가 확인 라벨(" `{총액} KRW {매수\|매도} 확정` ")로 전환, 3초 내 재클릭으로 확정한다. Esc·외부 클릭·3초 경과·입력(방향·유형·가격·수량·마켓) 변경 시 원복. 모달·바텀시트 없음(상주 폼의 포커스 이탈 비용 배제).

OrderbookPanel · TradesPanel. 등락률 조인

orderbook·trade 메시지에는 전일종가가 없다. `derive.ts` 가 `tickers[market]` 에서 역산한다: 전일종가 = 현재가 ∓ 등락액(RISE 차감·FALL 가산). 임의 가격의 등락률은 이 전일종가 대비로 계산해 각 호가·체결 행에 색·부호로 병기한다. 얇은 호가의 빈 슬롯(가격·잔량 0)은 등락률 오표시(-100%)를 막기 위해 값 없는 자리행으로 렌더한다.

상태 (로딩 · 빈 · 에러)

상태	처리
로딩	티커 미수신 시 SummaryBar 현재가 Skeleton, 호가·체결·차트 각 Skeleton(`variant` 대응)
빈	체결 없음 → "아직 체결 내역이 없어요". 마켓 목록 탭별 빈 문구(관심·보유·검색 결과)
존재하지 않는 마켓	`COIN_META` 미존재 → EmptyState + [거래소로 돌아가기]
연결 단절	HeaderNav 연결 점이 "연결 끊김"으로 전환(`connected=false`). 시뮬레이터는 항상 연결

5.5 온보딩. `/welcome`

데이터	없음 (정적)
이벤트	[시작하기]·[이미 계정이 있어요] 모두 `/exchange` . 완전 시뮬레이션이라 계정 생성·비밀번호·인증 UI 없음

5.2 자산. `/assets`

구조 · 컴포넌트

```
AssetsPage (max-width 1200px)
├ 총 보유자산 요약 카드 (전폭)  ── 총평가·총매수·평가손익(PricePill)·보유현금·총자산 · 숨김 토글
└ grid 1fr 320px
   ├ 보유 자산 테이블 (DataTable) ── 코인(SymbolChip)·보유수량·매수평균가·평가금액·평가손익, 행 48px
   └ 보유 비중 카드              ── 상위 5 + 기타 가로 막대(role="meter")
```

항목	규칙
데이터	`assets` ← `loadAccounts()` + `useLiveChannels([...보유 마켓 ticker, dest.myAsset])`
평가 재계산	보유량 = balance + locked. 평가손익 = (현재가 − 매수평균가)×수량, 수익률 = 손익/매수금액. selector 메모
손익 표시	색 + 부호 병기(`dir-text` + `formatDirectionalAmount` + 등락률). green 금지
숨김 토글	금액 전부 `••••••` 마스킹 (로컬 상태)
행 클릭	`/exchange/[market]` 이동
빈 상태	"아직 보유한 코인이 없어요" + [거래소 가기]

5.3 거래내역. `/history`

구조 · 컴포넌트

```
HistoryPage (max-width 1200px, 전폭 단일 컬럼)
├ 필터 바 (48px)  ── SegmentedTab(미체결·체결) + 마켓 셀렉트 + 기간 칩(전체·1일·1주·1개월)
└ 본문 테이블 (DataTable, 행 44px)
   시각 | 마켓(SymbolChip) | 구분(매수Red·매도Blue 배지) | 주문가격 | 주문수량 | 체결수량 | 상태 | [취소]
```

항목	규칙
데이터	`openOrders` / `closedOrders` ← `loadOrders()` + `useLiveChannels([dest.myOrder])`. 스토어 apply가 미체결↔체결 전이
상태 배지	부분체결(warning) · 체결(success) · 취소됨/접수(무채색). 색 도메인은 헬스, 방향 배지는 시세 색
취소	미체결 탭만 노출. `cancelOrder(order.id)` (UUID) → Toast + `loadOrders()`. 중복 클릭 차단(canceling 셋)
필터	마켓·기간 클라이언트 필터. 정렬은 DataTable(시각 desc 기본)
빈 상태	탭별 문구 + [거래소 가기]

5.4 시스템 상태. `/system`

구조 · 컴포넌트

```
SystemPage (max-width 1200px)
├ 전체 상태 배너 (전폭 72px)     ── 정상 success / 성능저하 warning · 가동률·처리량·p99·최근 RTO
├ 서비스 카드 3열 grid (각 140px) ── 노드명(mono)·상태점+라벨·지연시간·가동률
└ 이벤트 로그 테이블 (DataTable) ── 시각·서비스·수준·메시지, 최신순, 행 40px
```

항목	규칙
데이터(현재)	클라이언트 시뮬레이션(`advance` 1s 틱). 노드 6종(API·체결 엔진·저널 DB), 확률적 성능 저하 주입 후 5~10틱 자동 복구
색 도메인	시스템 헬스(success/warning/danger). 시세 색과 분리. 배지+아이콘+라벨 병기
상태 전이 플래시	헬스 전용 토큰(`.status-pill[data-transition]`). 시세 틱 플래시 차용 금지
복구 관찰	노드 복구 시 RTO 실측 갱신 + Toast(success)
데이터(Spring 전환 후)	헬스/메트릭 API 폴링(5s) + Prometheus 지표 미러로 교체(§6.1-7)

시스템 화면은 현재 프론트 내부 시뮬레이션이라 DataSource를 경유하지 않는다. 전환 시 별도 헬스/메트릭 소스로 교체한다(다른 화면의 무수정 원칙과 무관).

6. Spring 백엔드 연동 전환 계획

전환 원칙: 화면 코드 무수정. `HttpStompDataSource`(`lib/api/httpDatasource.ts`)가 `DataSource` 인터페이스를 구현하고 `getDataSource()` 팩토리가 `NEXT_PUBLIC_DATASOURCE` 로 선택한다. 어댑터는 이미 구현돼 있고 시뮬레이터와 동일 계약을 미러한다.

6.1 전환 체크리스트

#	항목	내용
1	환경 변수	`NEXT_PUBLIC_DATASOURCE=sim|http` (기본 sim), `NEXT_PUBLIC_API_BASE` , `NEXT_PUBLIC_WS_BASE`
2	REST 어댑터	§3.3 매핑대로 fetch. 리스트 엔벌로프(`{data}`)·표준 에러(`{error:{name,message,retry_after}}` → `ApiError`) 파싱. 금액은 **문자열 유지**(NUMERIC(40,18)). 캔들 조회 실패는 빈 배열로 흡수(표시 전용 피드)
3	STOMP 어댑터	`@stomp/stompjs` 로 `/ws/v1` 연결. `subscribe(dest, cb)` → SUBSCRIBE. 재연결 시 전 채널 재구독 + REST 재동기화. 신규 구독자에 초기 스냅샷 1건 합성 전달(`deliverInitialSnapshot`)
4	인증	데모 세션(`POST /api/v1/demo/session`) 단기 JWT를 메모리에만 보관. 401 시 재발급 후 1회 재시도. 비공개 채널 구독 시 Bearer CONNECT로 승격. **브라우저에 secret 저장 금지**
5	seq/epoch	채널별 seq 단조 검사·gap REST 재동기화·epoch 변경 강제 재동기화를 어댑터가 담당. 재동기화 중 도착분은 버퍼링 후 seq 오름차순 flush. store에는 연속화된 도메인 이벤트만 전달(raw `seq/epoch/stream` 미노출)
6	주문 멱등	`client_order_id` 생성·재시도 재사용은 OrderPanel에 구현 완료. 어댑터 무관. 취소는 `order.id` (UUID) 정규 라우트
7	시스템 화면	`/system` 데이터 소스를 내부 시뮬레이션 → 헬스/메트릭 API 폴링(5s)으로 교체
8	검증	전환 후 하네스 봇 플릿·k6 시나리오가 동일 API 사용. E2E 스모크: 마켓 조회 → 주문 → 체결 Toast → 거래내역 반영

6.2 백엔드에 요구되는 계약 (수용 기준 4가지)

> Spring 구현이 지켜야 프론트가 무수정으로 붙는 것: ① API v2.5의 REST 경로·엔벌로프·에러코드 ② WS 채널 경로(§5-3)· `seq/epoch/stream` 필드·구독 즉시 스냅샷 1건(§5-2) ③ `client_order_id` 멱등(§2-4) ④ 금액 문자열 표기(NUMERIC(40,18)). 이 4가지는 `SimDataSource` 가 이미 미러하고 `HttpStompDataSource` 가 검증하므로, **백엔드 구현의 수용 테스트 기준**으로 그대로 사용한다.

본 문서는 `프로젝트/web` 구현(데스크톱 그리드 + 서브 3화면 + 온보딩)과 1:1 정합하며, 이후 코드 변경 시 함께 갱신한다. 웹 화면 명세서 v2.0 끝.